

System Architect Role – Deploying the Multi-Sensor TTC Fusion Pipeline

2026-09-20 · @Someone

Role purpose and scope

The system architect turns your offline, notebook-based TTC fusion pipeline into a real-time, safety-assured on-vehicle function, and owns every cross-cutting decision that makes that possible.

Today the pipeline processes 404 recorded nuScenes-mini samples at 2 Hz in Python notebooks. The deployment target is a vehicle function that ingests live LiDAR, radar and camera streams, estimates time-to-collision per tracked object, and passes a validated result to a warning or braking (AEB) decision layer within a fixed latency budget.

The architect does not write every algorithm. They define the system boundaries, interfaces, timing, compute allocation, safety concept and validation plan, and they arbitrate trade-offs between perception, embedded, safety and test teams.

Assumption: the target is an ADAS-level function (driver-supervised, SAE Level 1 to 2), not driverless operation. This drives the safety integrity level and the validation effort, and should be confirmed early.

Where the prototype stands and the gaps

The prototype proves the concept offline, but its accuracy, coverage and timing are all short of what a vehicle function needs. The measured results below come from the Step 6 evaluation on nuScenes-mini.

Area	Prototype today	Gap to real deployment
Execution	Batch notebooks, Python, one step after another	Streaming, multi-threaded runtime with bounded latency
Frame rate	2 Hz keyframes (0.5 s step)	Sensor-rate operation, typically 10 to 20 Hz

Area	Prototype today	Gap to real deployment
Data	404 samples, 10 scenes, one region	Diverse ODD: weather, night, geography, traffic
TTC accuracy	Best MAE 2.74 s (camera, LiDAR depth), fused 3.80 s	Accuracy targets set per scenario, well below current error
Fusion benefit	Fused MAE is worse than LiDAR and camera	Fusion must beat the best single sensor to justify its cost
Coverage	14.3% of fused points combine 2+ sensors; 40 to 55% of points have a valid TTC	Defined behaviour when TTC is unavailable
Ground-truth match	Only 27% of fused tracks matched to annotation	Larger, fully labelled validation set
Repeatability	Two runs gave different track counts (for example 2,898 vs 2,838 LiDAR tracks)	Deterministic, versioned, reproducible outputs
Paths and config	Hard-coded Windows paths, files passed between notebooks	Configuration management and message-based interfaces
Safety	None: no fault handling, no fallback, no monitoring	Full safety concept, diagnostics and degraded modes

The first architectural task is to close the fusion gap: until fused TTC outperforms the best single sensor, the added complexity is hard to justify. Likely causes are the 71% single-sensor pass-through rate, the fragmented radar tracks (median length 2 frames), and the 60 s TTC cap.

Target system architecture

The target keeps your existing pipeline stages but runs them as a streaming graph with a shared clock, explicit interfaces and a safety monitor beside the main path.

flowchart LR

```

S[Sensors<br/>LiDAR, 5 radar, 6 camera] --> T[Time sync and<br/>ego-motion]
T --> P[Per-sensor perception<br/>ground removal, YOLO, radar filter]
P --> K[Per-sensor tracking<br/>global frame]
K --> F[Track fusion]
F --> C[TTC estimation<br/>UKF + CTRV]
C --> D[Decision layer<br/>warn or brake]
M[Safety monitor<br/>diagnostics, fallback] -.-> D
T -.-> M
F -.-> M

```

Read left to right: raw data is time-aligned first, perceived and tracked per sensor, fused, converted to TTC, and only then handed to the decision layer, with a monitor watching timing and plausibility throughout.

The architect decides the following layer by layer:

Layer	Prototype module	Architectural decision
Sensor interface	nuScenes files (Step 0, 1.x)	Drivers, hardware timestamps, PTP or GPS time sync, data rates
Ego-motion and calibration	Ego pose from dataset	Live IMU plus wheel odometry, online calibration monitoring
Perception	Steps 2.1 to 2.3	Which models run where, quantised YOLO, fixed-time ground removal
Tracking	Step 3	Frame-rate tracker with fixed memory, deterministic association
Fusion	Step 4	Track-level or object-level fusion, weighting by sensor health
TTC	Step 5	Real UKF with covariance output, uncertainty passed on with each TTC
Decision	Not present	Thresholds, hysteresis, HMI or brake request interface
Middleware	Files on disk	ROS 2 or AUTOSAR Adaptive, typed messages, no disk in the loop

One principle the architect should enforce: every TTC leaving the pipeline carries a validity flag and an uncertainty, so the decision layer never acts on a number it cannot trust.

Core responsibilities by workstream

The architect owns seven workstreams, each with a concrete output that other teams build against.

1. **Requirements and ODD.** Define the operational design domain (speed range, road types, weather, lighting) and derive measurable requirements: detection range, TTC accuracy per scenario, false-warning rate, latency. Output: system requirements specification with traceability to tests.
2. **Architecture and interfaces.** Specify the component split, message formats, timing contracts and error codes between sensors, perception, tracking, fusion, TTC and decision. Output: architecture description and interface control documents.
3. **Sensor suite and calibration.** Decide which sensors are mandatory and which are optional, mounting positions, field-of-view overlap and the calibration procedure (factory, service and online). Output: sensor placement study and calibration plan.
4. **Compute and software platform.** Choose the target SoC, operating system and middleware, allocate stages to CPU, GPU or accelerator, and set the software build and release process. Output: platform selection and resource budget.
5. **Safety and cybersecurity.** Lead hazard analysis, define safety goals, the fail-operational or fail-safe strategy and secure communication. Output: safety concept, threat analysis and risk assessment.
6. **Validation and verification.** Define the test pyramid from unit tests to simulation, closed track and public road, plus the metrics and acceptance thresholds. Output: verification and validation plan.
7. **Integration and lifecycle.** Coordinate integration with the vehicle network, manage variants, over-the-air updates and field monitoring after release. Output: integration plan and release process.

Across all seven, the architect keeps a single decision log so that each trade-off, for example accuracy against latency, has an owner, a rationale and a date.

Real-time, compute and hardware requirements

The prototype's slowest stage is LiDAR processing at about 1.4 s per sweep on a laptop CPU, so real-time operation needs a per-stage latency budget and hardware acceleration. The targets below are proposals for the architect to confirm, not measured values.

Stage	Prototype measurement	Proposed budget per cycle
Sensor capture and time sync	Not modelled	10 to 20 ms
LiDAR ground removal and clustering	About 1.4 s per sweep (CPU, Open3D and DBSCAN)	15 to 25 ms
Camera detection (6 cameras)	About 110 to 255 ms per image (CPU)	20 to 30 ms total, batched on GPU or NPU
Radar filtering	About 15 ms per sample	Under 5 ms
Tracking and fusion	About 50 ms per sample (fusion 18.5 s for 404 samples)	10 to 15 ms
UKF and TTC	Not timed separately	Under 5 ms
End-to-end (sensor to decision)	Not measured	100 ms or less, to be fixed against the AEB requirement

Key decisions for the architect:

- **Compute platform.** Pick an automotive-grade SoC with GPU or NPU for the camera network and vectorised LiDAR code. Size for the worst case: dense urban traffic with many clusters and tracks.
- **Port and optimise.** Rewrite hot paths in C++ or CUDA, replace DBSCAN with a grid or range-image method, and quantise YOLO to INT8 with accuracy checked after conversion.
- **Bound the worst case.** Cap points, clusters and tracks per frame so runtime is deterministic, and monitor deadline misses.
- **Memory and bandwidth.** A 32-beam LiDAR at about 35,000 points per sweep plus six camera streams needs a defined bus and memory budget.
- **Thermal and power.** Set limits early, since they constrain model size and frame rate.

The latency figures for radar and fusion are read from the notebook logs and are approximate; the architect should re-measure on target hardware.

Functional safety, standards and validation

A TTC function that can trigger braking must be developed as a safety-related system, so the architect needs a safety concept and a standards-based evidence trail before any road use. Standard editions and applicability should be confirmed with your safety and homologation teams.

Standard	What it covers	Relevance here
ISO 26262	Functional safety of electrical and electronic systems	Hazard analysis, ASIL rating (a warning-only function and an AEB function rate differently), safety goals
ISO 21448 (SOTIF)	Safety when the design is correct but performance is limited	Central for perception: missed detections, wrong TTC in rain, night or clutter
ISO/PAS 8800	Safety of AI in road vehicles	Covers the YOLO detector and any learned components
ISO/SAE 21434 and UN R155	Cybersecurity engineering and vehicle cybersecurity management	Sensor spoofing, secure messaging, update integrity
UN R156	Software update management	Over-the-air updates of models and parameters
UN R152 and Euro NCAP AEB protocols	Regulatory and rating tests for automatic emergency braking	Target scenarios and pass criteria if the function brakes
Automotive SPICE	Software process assessment	Traceable requirements, design, test and reviews

Safety design decisions the architect must make:

- **Fault detection.** Detect sensor blockage, calibration drift, timestamp jumps and out-of-range values, and reduce or disable the function when a fault is confirmed.

- **Degraded modes.** Define what happens when one sensor is lost. Because fusion may draw on LiDAR, radar or camera alone, each combination needs a defined accuracy claim.
- **Redundancy and plausibility.** Cross-check TTC between independent sensors and pass the disagreement as an uncertainty rather than hiding it.
- **Decision rules.** Use hysteresis, minimum persistence and confidence gating so a single noisy TTC value cannot trigger an action.

Validation should proceed through four levels:

1. **Software-in-the-loop** on recorded data: regression tests reproducing the Step 7 audit checks, run on every code change.
2. **Simulation** with scenario libraries (cut-in, cut-out, pedestrian crossing, stationary obstacle) and injected sensor faults.
3. **Closed track** with instrumented targets and reference GNSS to measure true TTC, replacing the annotation-derived ground truth.
4. **Public road** with shadow mode: the function runs and logs without acting, so false-warning rates can be measured over large distances.

Acceptance criteria should be stated per scenario, for example maximum TTC error within a given range and maximum false-warning rate per 1,000 km, once the requirements in workstream 1 are agreed.

Data, calibration and model lifecycle

The architect must build a data and update loop, because a perception system trained and tuned on 404 samples will fail on conditions it has never seen.

- **Training and tuning data.** Collect a vehicle-specific dataset covering the ODD, including rain, night, glare, tunnels and dense traffic. Label with 3D boxes and object identities so TTC can be scored directly.
- **Detector adaptation.** The prototype uses a generic pretrained YOLO with fixed class heights for monocular depth (car 1.5 m, person 1.7 m). Fine-tune on target-domain data and replace fixed heights with learned depth or LiDAR-assisted depth.
- **Parameter tuning.** Thresholds such as the 3.0 m association gate, 2.5 m merge distance, RANSAC 0.2 m and DBSCAN eps 0.7 m were tuned on one dataset. Re-tune per sensor model and mounting, and put them under configuration control.

- **Calibration management.** Define factory calibration, end-of-line checks, service recalibration and runtime checks that flag drift, since a small extrinsic error shifts LiDAR-to-camera projection and corrupts fused positions.
- **Ground truth for evaluation.** Move from annotation-derived TTC to reference-instrumented drives, so the MAE and RMSE figures reflect real vehicle behaviour.
- **Model and software versioning.** Version datasets, models, parameters and code together so any result can be reproduced. This also fixes the run-to-run differences seen in the notebooks by seeding RANSAC and DBSCAN or replacing them with deterministic methods.
- **Fleet monitoring.** After release, log low-confidence events and near-misses in privacy-compliant form, and use them to drive retraining and to detect drift.

An update path (over-the-air, with rollback and safety re-approval) should be designed at the start, not added later.

Phased roadmap, risks and open decisions

Deployment should proceed in four phases, each ending with a decision gate that the architect prepares and presents. Durations are indicative and depend on team size.

Phase	Focus	Key deliverables	Exit gate
1. Foundation	Requirements, ODD, safety concept, fix fusion gap and determinism	Requirements spec, architecture v1, reproducible pipeline	Fused TTC beats best single sensor on a larger dataset
2. Real-time port	Streaming runtime, C++ and GPU ports, time sync	Middleware design, latency measurements on target SoC	End-to-end latency within budget
3. Vehicle integration	Sensor install, calibration, fault handling, decision layer	Prototype vehicle, degraded-mode tests, HMI or brake interface	Track and simulation tests passed
4. Validation and release	Shadow mode, road tests, homologation evidence	Safety case, V&V report, release process	Safety and regulatory sign-off

The main risks and their mitigations:

Risk	Why it matters	Mitigation
Fusion does not improve accuracy	Undermines the multi-sensor business case	Diagnose track fragmentation and single-sensor pass-through first
Latency exceeds budget	Late warnings reduce safety value	Early profiling on target hardware, hard caps on load
Perception failure in unseen conditions	Missed or false TTC	SOTIF analysis, target-domain data, shadow mode
Calibration drift	Corrupts projection and fusion	Runtime plausibility checks and recalibration procedure
Radar track fragmentation	Weak radar contribution (MAE 5.54 s)	Improve association and stationary-object handling
Scope creep to higher automation	Raises safety level and cost sharply	Freeze ODD and function level at Phase 1

Open decisions to settle first:

- ☐ Function level: warning only, or warning plus automatic braking
- ☐ ODD: speed range, road types, weather and lighting conditions
- ☐ Sensor set: is radar or LiDAR mandatory in series production, given cost
- ☐ Target hardware platform and middleware
- ☐ Accuracy and false-warning targets per scenario
- ☐ Regions and regulations for homologation